

Universidad Autonoma de Nuevo Leon

Equipo:

Eduardo Zavala Ontiveros

Axel Ivan Vearstegui Patino

Jaime Cesar Zermeno Pereyra

Sistema de Ventas con Stored Procedures y Triggers

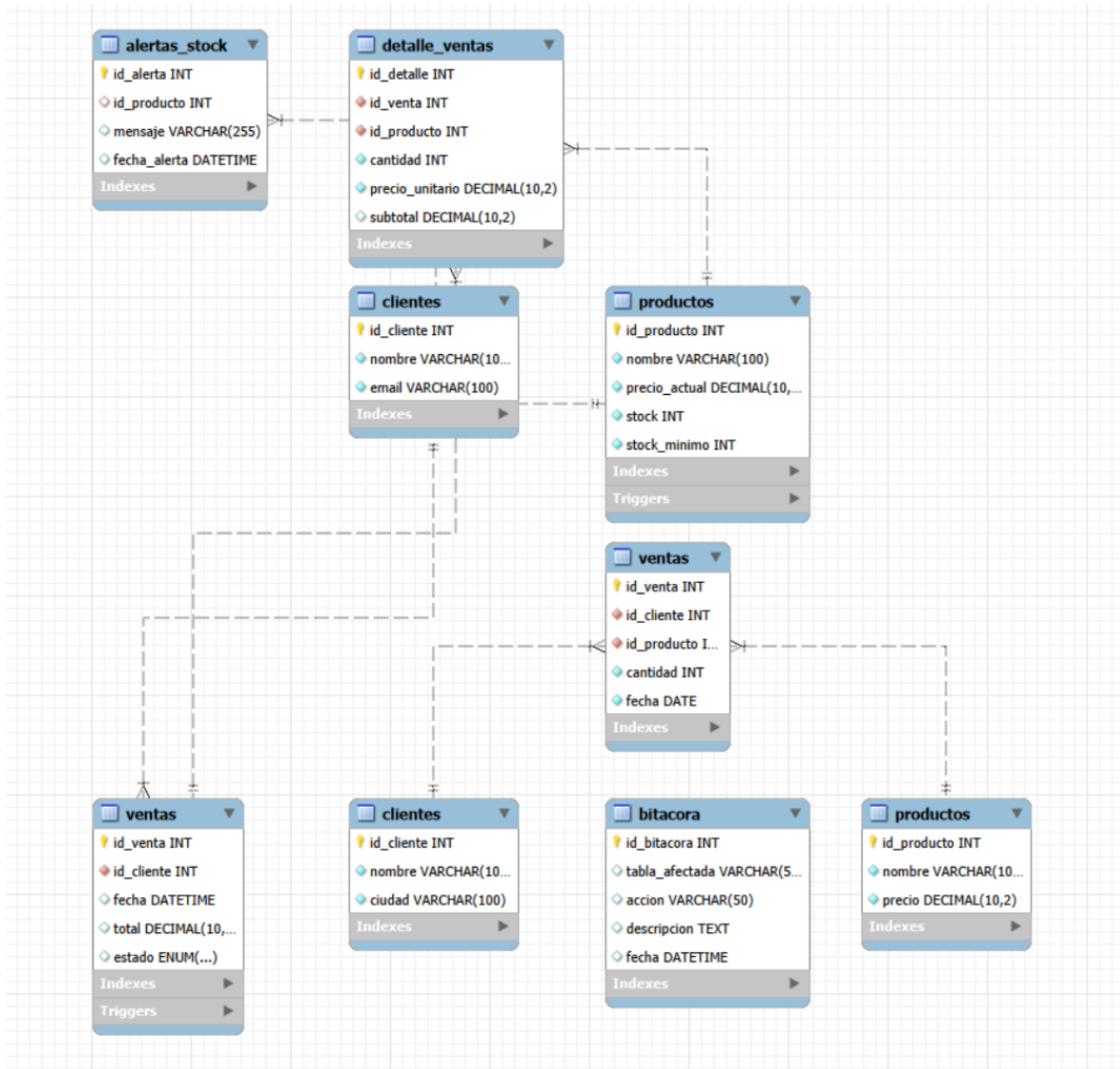
Facultad de Ciencias Fisico Matematicas

Bases de Datos

Docente:

ISLAS PINEDA JORGE ALBERTO

Modelo de entidad relación:



Normalizacion:

Para que el modelo alcance la 3FN, cada tabla debe cumplir con lo siguiente:

1 FN: Datos atómicos, sin grupos repetidos (por eso separamos Ventas y Detalle_Ventas).

2 FN: Cumple la 1FN y todos los atributos no clave dependen de la clave primaria completa (el precio histórico se guarda en el detalle, no se depende del precio actual del producto).

3FN: Cumple la 2FN y no existen dependencias transitivas (ningún atributo no clave depende de otro atributo no clave).

Script (Fotos):

Creación e inserciones

```
4 • CREATE DATABASE IF NOT EXISTS SistemaVentas_DB;
5 • USE SistemaVentas_DB;
6
7 • CREATE TABLE Clientes (
8     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
9     nombre VARCHAR(100) NOT NULL,
10    email VARCHAR(100) UNIQUE NOT NULL
11 );
12
13 • CREATE TABLE Productos (
14     id_producto INT AUTO_INCREMENT PRIMARY KEY,
15     nombre VARCHAR(100) NOT NULL,
16     precio_actual DECIMAL(10,2) NOT NULL,
17     stock INT NOT NULL,
18     stock_minimo INT NOT NULL DEFAULT 5
19 );
20
21 • CREATE TABLE Ventas (
22     id_venta INT AUTO_INCREMENT PRIMARY KEY,
23     id_cliente INT NOT NULL,
24     fecha DATETIME DEFAULT CURRENT_TIMESTAMP,
25     total DECIMAL(10,2) DEFAULT 0.00,
26     estado ENUM('ACTIVA', 'CANCELADA') DEFAULT 'ACTIVA',
27     FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
28 );
29
30 • CREATE TABLE Detalle_Ventas (
31     id_detalle INT AUTO_INCREMENT PRIMARY KEY,
32     id_venta INT NOT NULL,
33     id_producto INT NOT NULL,
34     cantidad INT NOT NULL,
35     precio_unitario DECIMAL(10,2) NOT NULL,
36     subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED,
37     FOREIGN KEY (id_venta) REFERENCES Ventas(id_venta),
38     FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)
39 );
40
```

```
41 • CREATE TABLE Bitacora (  
42     id_bitacora INT AUTO_INCREMENT PRIMARY KEY,  
43     tabla_afectada VARCHAR(50),  
44     accion VARCHAR(50),  
45     descripcion TEXT,  
46     fecha DATETIME DEFAULT CURRENT_TIMESTAMP  
47 );  
48  
49 • CREATE TABLE Alertas_Stock (  
50     id_alerta INT AUTO_INCREMENT PRIMARY KEY,  
51     id_producto INT,  
52     mensaje VARCHAR(255),  
53     fecha_alerta DATETIME DEFAULT CURRENT_TIMESTAMP,  
54     FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)  
55 );  
56  
57 -- insercion de Datos  
58 • INSERT INTO Clientes (nombre, email) VALUES  
59 ('Alan Turing', 'alan.t@ejemplo.com'),  
60 ('Ada Lovelace', 'ada.l@ejemplo.com'),  
61 ('Donald Knuth', 'd.knuth@ejemplo.com');  
62  
63 • INSERT INTO Productos (nombre, precio_actual, stock, stock_minimo) VALUES  
64 ('Laptop Thinkpad', 15000.00, 10, 3),  
65 ('Teclado Mecánico', 1200.00, 20, 5),  
66 ('Monitor 27 Pulgadas', 4500.00, 6, 2);  
67
```

Triggers:

```
3 DELIMITER //
4 • -- trigger 1 alerta de Stock
5 CREATE TRIGGER trg_alerta_stock
6 AFTER UPDATE ON Productos
7 FOR EACH ROW
8 BEGIN
9     IF NEW.stock <= NEW.stock_minimo AND OLD.stock > OLD.stock_minimo THEN
10         INSERT INTO Alertas_Stock (id_producto, mensaje)
11         VALUES (NEW.id_producto, CONCAT('Atención: El stock ha caído a ', NEW.stock, ' unidades.'));
12     END IF;
13 END //
14
15 -- Trigger 2 Bitacora de Cambios
16 • CREATE TRIGGER trg_bitacora_ventas
17 AFTER UPDATE ON Ventas
18 FOR EACH ROW
19 BEGIN
20     IF NEW.estado = 'CANCELADA' AND OLD.estado = 'ACTIVA' THEN
21         INSERT INTO Bitacora (tabla_afectada, accion, descripcion)
22         VALUES ('Ventas', 'CANCELACION', CONCAT('Se canceló la venta ID: ', NEW.id_venta, ' por un monto de $', NEW.total));
23     END IF;
24 END //
25
26 DELIMITER ;
27
```

Store Procedures:

```
5 CREATE PROCEDURE sp_registrar_venta (
6     IN p_id_cliente INT,
7     IN p_id_producto INT,
8     IN p_cantidad INT
9 )
10 BEGIN
11     DECLARE v_precio DECIMAL(10,2);
12     DECLARE v_stock_actual INT;
13     DECLARE v_id_venta INT;
14
15     DECLARE EXIT HANDLER FOR SQLEXCEPTION
16     BEGIN
17         ROLLBACK;
18         RESIGNAL;
19     END;
20
21     START TRANSACTION;
22
23     -- verificar stock y obtener precio
24     SELECT precio_actual, stock INTO v_precio, v_stock_actual
25     FROM Productos WHERE id_producto = p_id_producto FOR UPDATE;
26
27     IF v_stock_actual < p_cantidad THEN
28         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Stock insuficiente para realizar la venta.';
29     ELSE
30         INSERT INTO Ventas (id_cliente, total, estado) VALUES (p_id_cliente, v_precio * p_cantidad, 'ACTIVA');
31         SET v_id_venta = LAST_INSERT_ID();
32
33         INSERT INTO Detalle_Ventas (id_venta, id_producto, cantidad, precio_unitario)
34         VALUES (v_id_venta, p_id_producto, p_cantidad, v_precio);
35
36         UPDATE Productos SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;
37
38         COMMIT;
39     END IF;
40 END //

42 -- SP cancelar venta
43 CREATE PROCEDURE sp_cancelar_venta (
44     IN p_id_venta INT
45 )
46 BEGIN
47     DECLARE v_estado_actual VARCHAR(20);
48
49     DECLARE EXIT HANDLER FOR SQLEXCEPTION
50     BEGIN
51         ROLLBACK;
52         RESIGNAL;
53     END;
54
55     START TRANSACTION;
56
57     SELECT estado INTO v_estado_actual FROM Ventas WHERE id_venta = p_id_venta FOR UPDATE;
58
59     IF v_estado_actual = 'CANCELADA' THEN
60         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: La venta ya se encuentra cancelada.';
61     ELSE
62         UPDATE Productos p
63         INNER JOIN Detalle_Ventas dv ON p.id_producto = dv.id_producto
64         SET p.stock = p.stock + dv.cantidad
65         WHERE dv.id_venta = p_id_venta;
66
67         UPDATE Ventas SET estado = 'CANCELADA' WHERE id_venta = p_id_venta;
68
69         COMMIT;
70     END IF;
71 END //
```

```
74 ● ○ CREATE PROCEDURE sp_reporte_ventas (  
75     IN p_fecha_inicio DATE,  
76     IN p_fecha_fin DATE  
77 )  
78 ○ BEGIN  
79     SELECT  
80         v.id_venta,  
81         c.nombre AS Cliente,  
82         v.fecha,  
83         v.total,  
84         v.estado,  
85         COUNT(dv.id_detalle) AS articulos_diferentes  
86     FROM Ventas v  
87     JOIN Clientes c ON v.id_cliente = c.id_cliente  
88     LEFT JOIN Detalle_Ventas dv ON v.id_venta = dv.id_venta  
89     WHERE DATE(v.fecha) BETWEEN p_fecha_inicio AND p_fecha_fin  
90     GROUP BY v.id_venta  
91     ORDER BY v.fecha DESC;  
92     END //  
93  
94     DELIMITER ;  
95
```


Llamando al SP c

- `CALL sp_registrar_venta(1, 1, 2);`

✓	16	11:28:16	CALL sp_registrar_venta(1, 1, 2)
---	----	----------	----------------------------------

```
SELECT * FROM Ventas;
```

Result Grid		Filter Rows:		Edit:		Export
	id_venta	id_cliente	fecha	total	estado	
▶	1	1	2026-05-26 11:28:16	30000.00	ACTIVA	
*	NULL	NULL	NULL	NULL	NULL	

```
SELECT * FROM Detalle_Ventas;
```

Result Grid		Filter Rows:		Edit:		Export
	id_detalle	id_venta	id_producto	cantidad	precio_unitario	subtotal
▶	1	1	1	2	15000.00	30000.00
*	NULL	NULL	NULL	NULL	NULL	NULL

```
SELECT id_producto, nombre, stock FROM Productos WHERE id_producto = 1;
```

	id_producto	nombre	stock
▶	1	Laptop Thinkpad	8
*	NULL	NULL	NULL

Testeo 2: Con alerta de Stock y cancelando venta

```
CALL sp_registrar_venta(2, 1, 5);
```

✓ 23 11:48:14 CALL sp_registrar_venta(2, 1, 5)

```
SELECT * FROM Alertas_Stock;
```

Result Grid				
		Filter Rows:		Edit: 
				Export/Import: 
	id_alerta	id_producto	mensaje	fecha_alerta
▶	1	1	Atención: El stock ha caído a 3 unidades.	2026-05-26 11:48:14
*	NULL	NULL	NULL	NULL

```
CALL sp_cancelar_venta(1);
```

✓ 25 11:49:11 CALL sp_cancelar_venta(1)

```
SELECT id_venta, estado FROM Ventas WHERE id_venta = 1;
```

	id_venta	estado
▶	1	CANCELADA
*	NULL	NULL

Testeo 3: Usando bitácora y el reporte de ventas:

```
SELECT id_producto, stock FROM Productos WHERE id_producto = 1;
```

Result Grid			Filter Rows:
	id_producto	stock	
▶	1	5	
★	NULL	NULL	

```
SELECT * FROM Bitacora;
```

	id_bitacora	tabla_afectada	accion	descripcion	fecha
▶	1	Ventas	CANCELACION	Se canceló la venta ID: 1 por un monto de \$300...	2026-05-26 11:49:12
★	NULL	NULL	NULL	NULL	NULL

```
CALL sp_reporte_ventas(CURRENT_DATE, CURRENT_DATE);
```

	id_venta	Cliente	fecha	total	estado	articulos_diferentes
▶	2	Ada Lovelace	2026-05-26 11:48:14	75000.00	ACTIVA	1
	1	Alan Turing	2026-05-26 11:28:16	30000.00	CANCELADA	1

Codigo de SQL:

```
-- tablas y BD
```

```
CREATE DATABASE IF NOT EXISTS SistemaVentas_DB;
```

```
USE SistemaVentas_DB;
```

```
CREATE TABLE Clientes (
```

```
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) UNIQUE NOT NULL
```

```
);
```

```
CREATE TABLE Productos (
```

```
    id_producto INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    precio_actual DECIMAL(10,2) NOT NULL,
```

```
    stock INT NOT NULL,
```

```
    stock_minimo INT NOT NULL DEFAULT 5
```

```
);
```

```
CREATE TABLE Ventas (
```

```
    id_venta INT AUTO_INCREMENT PRIMARY KEY,
```

```
    id_cliente INT NOT NULL,
```

```

    fecha DATETIME DEFAULT CURRENT_TIMESTAMP,

    total DECIMAL(10,2) DEFAULT 0.00,

    estado ENUM('ACTIVA', 'CANCELADA') DEFAULT 'ACTIVA',

    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)

);

```

```

CREATE TABLE Detalle_Ventas (

    id_detalle INT AUTO_INCREMENT PRIMARY KEY,

    id_venta INT NOT NULL,

    id_producto INT NOT NULL,

    cantidad INT NOT NULL,

    precio_unitario DECIMAL(10,2) NOT NULL,

    subtotal DECIMAL(10,2) GENERATED ALWAYS AS (cantidad *
precio_unitario) STORED,

    FOREIGN KEY (id_venta) REFERENCES Ventas(id_venta),

    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)

);

```

```

CREATE TABLE Bitacora (

    id_bitacora INT AUTO_INCREMENT PRIMARY KEY,

    tabla_afectada VARCHAR(50),

    accion VARCHAR(50),

    descripcion TEXT,

```

```

        fecha DATETIME DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE Alertas_Stock (
    id_alerta INT AUTO_INCREMENT PRIMARY KEY,
    id_producto INT,
    mensaje VARCHAR(255),
    fecha_alerta DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)
);

-- insercion de Datos

INSERT INTO Clientes (nombre, email) VALUES
('Alan Turing', 'alan.t@ejemplo.com'),
('Ada Lovelace', 'ada.l@ejemplo.com'),
('Donald Knuth', 'd.knuth@ejemplo.com');

INSERT INTO Productos (nombre, precio_actual, stock, stock_minimo)
VALUES
('Laptop Thinkpad', 15000.00, 10, 3),
('Teclado Mecánico', 1200.00, 20, 5),
('Monitor 27 Pulgadas', 4500.00, 6, 2);

```

```
-- triggers
```

```
DELIMITER //
```

```
-- trigger 1 alerta de Stock
```

```
CREATE TRIGGER trg_alerta_stock
```

```
AFTER UPDATE ON Productos
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF    NEW.stock    <=    NEW.stock_minimo    AND    OLD.stock    >  
    OLD.stock_minimo THEN
```

```
        INSERT INTO Alertas_Stock (id_producto, mensaje)
```

```
        VALUES (NEW.id_producto, CONCAT('Atención: El stock ha caído  
a ', NEW.stock, ' unidades.'));
```

```
    END IF;
```

```
END //
```

```
-- Trigger 2 Bitacora de Cambios
```

```
CREATE TRIGGER trg_bitacora_ventas
```

```
AFTER UPDATE ON Ventas
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.estado = 'CANCELADA' AND OLD.estado = 'ACTIVA' THEN
```

```
        INSERT INTO Bitacora (tabla_afectada, accion, descripcion)
```

```
VALUES ('Ventas', 'CANCELACION', CONCAT('Se canceló la venta
ID: ', NEW.id_venta, ' por un monto de $', NEW.total));
```

```
END IF;
```

```
END //
```

```
-- Stores procedures
```

```
DELIMITER //
```

```
-- SP 1 Registrar Venta
```

```
CREATE PROCEDURE sp_registrar_venta (
```

```
    IN p_id_cliente INT,
```

```
    IN p_id_producto INT,
```

```
    IN p_cantidad INT
```

```
)
```

```
BEGIN
```

```
    DECLARE v_precio DECIMAL(10,2);
```

```
    DECLARE v_stock_actual INT;
```

```
    DECLARE v_id_venta INT;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
    BEGIN
```

```
        ROLLBACK;
```

```
        RESIGNAL;
```


END;

START TRANSACTION;

-- verificar stock y obtener precio

SELECT precio_actual, stock INTO v_precio, v_stock_actual

FROM Productos WHERE id_producto = p_id_producto FOR UPDATE;

IF v_stock_actual < p_cantidad THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Stock
insuficiente para realizar la venta.';

ELSE

INSERT INTO Ventas (id_cliente, total, estado) VALUES
(p_id_cliente, v_precio * p_cantidad, 'ACTIVA');

SET v_id_venta = LAST_INSERT_ID();

INSERT INTO Detalle_Ventas (id_venta, id_producto, cantidad,
precio_unitario)

VALUES (v_id_venta, p_id_producto, p_cantidad, v_precio);

UPDATE Productos SET stock = stock - p_cantidad WHERE
id_producto = p_id_producto;

```

        COMMIT;

    END IF;

END //
```

-- SP cancelar venta

```

CREATE PROCEDURE sp_cancelar_venta (

    IN p_id_venta INT

)

BEGIN

    DECLARE v_estado_actual VARCHAR(20);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    START TRANSACTION;

    SELECT estado INTO v_estado_actual FROM Ventas WHERE id_venta =

p_id_venta FOR UPDATE;

    IF v_estado_actual = 'CANCELADA' THEN
```

```
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: La venta  
ya se encuentra cancelada.';
```

```
ELSE
```

```
UPDATE Productos p
```

```
INNER JOIN Detalle_Ventas dv ON p.id_producto =  
dv.id_producto
```

```
SET p.stock = p.stock + dv.cantidad
```

```
WHERE dv.id_venta = p_id_venta;
```

```
UPDATE Ventas SET estado = 'CANCELADA' WHERE id_venta =  
p_id_venta;
```

```
COMMIT;
```

```
END IF;
```

```
END //
```

```
-- SP 3: Reporte de Ventas por Rango de Fechas
```

```
CREATE PROCEDURE sp_reporte_ventas (
```

```
IN p_fecha_inicio DATE,
```

```
IN p_fecha_fin DATE
```

```
)
```

```
BEGIN
```

```
SELECT
```

```

        v.id_venta,

        c.nombre AS Cliente,

        v.fecha,

        v.total,

        v.estado,

        COUNT(dv.id_detalle) AS articulos_diferentes

FROM Ventas v

JOIN Clientes c ON v.id_cliente = c.id_cliente

LEFT JOIN Detalle_Ventas dv ON v.id_venta = dv.id_venta

WHERE DATE(v.fecha) BETWEEN p_fecha_inicio AND p_fecha_fin

GROUP BY v.id_venta

ORDER BY v.fecha DESC;

END //

DELIMITER ;

-- Test cases

CALL sp_registrar_venta(1, 1, 2);

SELECT * FROM Ventas;

SELECT * FROM Detalle_Ventas;

```

```
SELECT id_producto, nombre, stock FROM Productos WHERE id_producto  
= 1;
```

```
CALL sp_registrar_venta(2, 1, 5);
```

```
SELECT * FROM Alertas_Stock;
```

```
CALL sp_cancelar_venta(1);
```

```
SELECT id_venta, estado FROM Ventas WHERE id_venta = 1;
```

```
SELECT id_producto, stock FROM Productos WHERE id_producto = 1;
```

```
-- Validar disparo del trigger de auditoría
```

```
SELECT * FROM Bitacora;
```

```
CALL sp_reporte_ventas(CURRENT_DATE, CURRENT_DATE);
```